UNIT 3

Employing hadoop mapreduce

Hadoop MapReduce is a core software framework used to process massive datasets in a distributed, parallel manner across thousands of commodity servers in a cluster. While the Hadoop Distributed File System (HDFS) handles the storage, MapReduce acts as the system's computation and processing engine**.**

Components of Hadoop MapReduce Jobs & Workflows

A MapReduce computation processes unstructured or semi-structured data by transforming key-value pairs across a distributed cluster.

Driver Class: Configures job options (input/output paths, format types, mapper/reducer implementations) and submits the job to the cluster.

Mapper Class: Implements the map() method. Converts raw input key-value pairs into zero or more intermediate key-value pairs.

Reducer Class: Implements the reduce() method. Aggregates values associated with each unique key

Combiner (Optional): Operates as a local reducer on the mapper node to reduce network traffic during the shuffle phase.

Partitioner: Determines which Reducer task receives a specific intermediate key based on its hash value.

Mapreduce Workflow

MapReduce Job WorkflowJob Submission: Client submits the job includes JAR file, Mapper & Reducer logic, input path and output path.

Task Assignment: The master node splits the job into smaller tasks and assigns them to worker nodes (TaskTrackers or YARN containers) based on data locality.

Map Execution: Mappers process input splits in parallel and write intermediate files.

Shuffle Phase: Intermediate data is sorted, grouped, and moved to the reduce nodes.

Reduce Execution: Reducers process the grouped data and write the final output to HDFS.

Completion: The application reports status updates until all tasks finish and the job completes.

Unit Testing with MR Unit & Local Testing

Testing distributed applications locally accelerates development by verifying business logic before cluster execution.

MRUnit: A JUnit-based testing framework designed specifically for unit testing Hadoop Mappers, Reducers, and Driver classes without starting a full Hadoop daemon.

MapDriver: Tests standalone Mapper logic.

ReduceDriver: Tests standalone Reducer logic.

MapReduceDriver: Tests Mapper and Reducer execution end-to-end.

Local Tests (Local JobRunner):

Running MapReduce jobs in-process using the local file system. Configuring mapreduce.framework.name=local allows developers to step through code using standard IDE debuggers before submitting to YARN or Classic MapReduce clusters.

Anatomy of a Classic MapReduce Job Run

Job Submission: The client program creates a job and sends it to the JobTracker (the master node).

Job Initialization: The JobTracker checks the input files, calculates input splits, and builds a task list (map and reduce tasks).

Task Assignment: TaskTrackers (worker nodes) send heartbeat signals to the JobTracker to ask for work.The JobTracker assigns tasks to idle TaskTrackers close to the data.

Mapping Phase: The TaskTracker runs the map task, reads data splits, and writes intermediate results to local hard drives.

Shuffle and Sort: Intermediate map data is copied over the network to the nodes running the reduce tasks, where it is grouped and sorted by key.

Reducing Phase: The reducer processes the sorted key-value pairs and writes the final output safely into the HDFS (Hadoop Distributed File System).

Distinguishing Hadoop Daemons: Classic vs. YARN

Feature	**Classic MapReduce (MR1)**	**YARN Architecture (MR2)**
Master node daemon	JobTracker(manages both resources & scheduling)	Resource Manager(manages cluster resources only)
Worker node daemon	TaskTracker(executes tasks in fixed slots)	Node Manager(Manages dynamic containers)
Job control	Centralized via jobtracker	Decentralized via per application ApplicationMaster
Scalability Limit	4,000 nodes per cluster	10,000+ nodes per cluster

Anatomy of a YARN Job Run

The lifecycle of a MapReduce job execution under YARN follows a 5-stage submission and execution cycle:

Job Submission: The client requests an Application ID from the ResourceManager and uploads job resources (JAR, configuration) to HDFS.

Job Initialization: ResourceManager allocates a container; the NodeManager starts the ApplicationMaster (AM).

Task Assignment: AM requests containers from the ResourceManager for Map and Reduce tasks based on data locality.

Task Execution: NodeManager spawns JVM containers to execute Map or Reduce tasks.

Completion: AM reports job status to the user and cleans up container resources.

YARN (Yet Another Resource Negotiator)

Decoupled Architecture: Separates cluster resource management from the programming model (MapReduce is now just one application type on YARN).

Container Concept: A logical bundle of resources (CPU and memory) allocated to run a specific task.

Scalability: Allows non-MapReduce frameworks (like Spark or Flink) to run on the same Hadoop cluster.

Failures in Classic

MapReduceTask Failure: A map or reduce task throws an error or the JVM crashes. The TaskTracker reports this failure to the JobTracker, which restarts the task up to four times on any available node.

TaskTracker Failure: A TaskTracker stops sending heartbeat signals. The JobTracker waits for a timeout period, assumes the node is dead, and reschedules all unfinished tasks that ran on that node.

JobTracker Failure: The JobTracker crashes or shuts down. This causes a single point of failure in classic Hadoop, immediately killing all running jobs and requiring a manual restart of the cluster.

Job Scheduling Strategies

To manage multi-tenant workloads, Hadoop provides three primary schedulers:

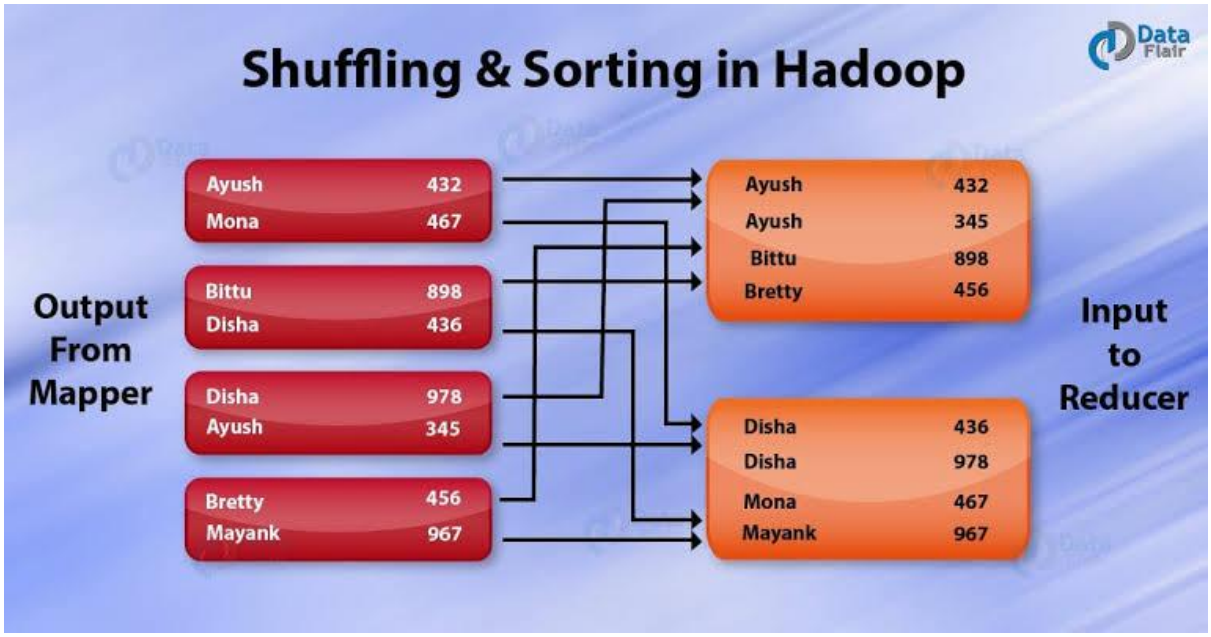
FIFO Scheduler: Jobs run strictly in order of submission. Simple, but large jobs block smaller ones.

Fair Scheduler: Allocates resources dynamically so all running jobs receive an equal share of memory and CPU capacity over time.

Capacity Scheduler: Allocates distinct queues with guaranteed minimum capacities for different teams or workloads, permitting excess resource sharing when available.

Shuffle and Sort Phase

The Shuffle and Sort phase bridges the Mapper and Reducer stages. It guarantees that all values associated with a single key arrive at the same Reducer in sorted order.



Steps in the Process:

Map Output Buffering: Mappers write output key-value pairs to a circular memory buffer (default size: 100MB).

Spilling: When the buffer reaches its threshold (typically 80%), data is partitioned, sorted in memory by key, and spilled to local disk.

Merging: Individual spill files on the local mapper disk are merged into a single sorted, partitioned file.

Shuffling (Copy Phase): Reducers fetch their specific key partitions from each Mapper's local disk over HTTP.

Sorting/Grouping: Reducer merges fetched partitions so keys are processed sequentially in a single reduce() call.

Task Execution Details

Speculative Execution: Hadoop identifies slow-running tasks ("stragglers") caused by hardware bottlenecks and launches identical duplicate tasks on alternate nodes. Whichever task finishes first commits its output; the slower instance is killed.

JVM Reuse: Enables a single container JVM to execute multiple sequential tasks, avoiding process launch overhead for small tasks.

Output Committers: Ensures atomic writes to HDFS. Output is staged in temporary directories (_temporary) and promoted to final output upon successful task execution.

MapReduce Types, Input, and Output Formats

Common Input Formats (InputFormat)

TextInputFormat: Default format; keys are byte offsets (LongWritable), values are line contents (Text).

KeyValueTextInputFormat: Treats the first delimiter character in each line as the key and the rest as the value.

NLineInputFormat: Splits input so each mapper receives a fixed number (N) of lines regardless of block size.

SequenceFileInputFormat: High-performance binary format optimized for storing sequence key-value structures.

Common Output Formats (OutputFormat)

TextOutputFormat: Writes key-value pairs as tab-separated plain text lines.

SequenceFileOutputFormat: Writes binary key-value pairs directly for downstream MapReduce consumption.

NullOutputFormat: Suppresses file creation (useful when writing directly to external databases like HBase or MongoDB).

UNIT 4

Apache Hbase

Apache hbase is a distributed, scalable, NoSQL big data store built on top of the Hadoop Distributed File System (HDFS).

Apache HBase is an open-source, non-relational, distributed column-oriented database modeled after Google's Bigtable. It runs on top of the Hadoop Distributed File System (HDFS) and provides real-time, random read/write access to large datasets.

Data Model and Implementations

Unlike traditional Relational Database Management Systems (RDBMS) that store data in fixed rows and columns, HBase organizes data into a multidimensional, sorted map:

{Data Mapping} = {RowKey , Column Family ,Column Qualifier ,Timestamp} →{Value}

- **Table:** A collection of rows sorted by the RowKey.
- **RowKey:** The unique primary key (byte array) used to index and retrieve records. Rows are lexicographically sorted by RowKey.
- **Column Family:** A logical group of related columns stored physically together on disk. Column families must be defined at table creation.
- **Column Qualifier:** The dynamic key within a column family (e.g., `info:name`, `info:email`).
- **Cell:** The intersection of a RowKey, Column Family, and Column Qualifier, containing a value and a 64-bit system timestamp for versioning.
- **Timestamp:** Enables versioning; HBase keeps multiple versions of data, returning the latest by default.

Architectural Components & Implementation

HMaster: Coordinates cluster operations, handles DDL operations (CREATE, DELETE table), and assigns RegionServers.

RegionServer: Handles data reads, writes, and storage management for a subset of rows (called Regions).

MemStore: In-memory write buffer for a region.

HFile: Immutable files stored in HDFS where flushed MemStore data resides (SSTable format).

Zookeeper: Maintains cluster coordination, active master election, and region location tracking.

HBase Clients

HBase provides several client interfaces for programmatic data access:

- Java API: The native client (`Org.apache.hadoop.hbase.client`) providing low-latency operations (`Table`, `Connection`, `Get`, `Put`, `Scan`, `Delete`).
- REST API (Stargate): Allows non-Java applications to query HBase via HTTP GET/POST/DELETE.
- Thrift API: Enables cross-language client bindings (Python, C++, PHP, Go).
- HBase Shell: Interactive JRuby-based command-line tool.

Praxis

RowKey Design: Avoid monotonically increasing RowKeys (e.g., sequential timestamps or IDs) to prevent Hotspotting on a single RegionServer. Use salted keys, hashed prefixes, or reversed timestamps.

Column Family Minimization: Keep column family counts low (1–3 per table). Multiple column families can lead to inefficient flushing mechanisms.

Pre-Splitting: Manually define region splits during table creation to distribute load evenly across servers.

Apache Cassandra

Apache Cassandra is a highly scalable, distributed NoSQL database designed to handle large amounts of data across multiple commodity servers with zero single point of failure (peer-to-peer masterless architecture).

Data Model

Cassandra uses a Distributed Hash Table (DHT) architecture based on Amazon DynamoDB and Google Bigtable concepts.

Keyspace: Top-level namespace, equivalent to an RDBMS Database schema. Defines replication settings (Replication Factor and Strategy).

Table (Column Family): Schema defined by a set of columns, containing rows.

Primary Key: Formed by two components:

Partition Key: Determines which node in the cluster stores the data via consistent hashing.

Clustering Key: Controls the physical sorting order of rows within a partition.

Cassandra Example

```
-- Insert operational data

INSERT INTO mca_dept.students (dept_id, roll_no, name, cgpa)

VALUES (101, 1, 'Rohan', 8.9);

INSERT INTO mca_dept.students (dept_id, roll_no, name, cgpa)

VALUES (101, 2, 'Priya', 9.2);

-- Query data (Must include Partition Key 'dept_id')

SELECT * FROM mca_dept.students WHERE dept_id = 101;
```

Cassandra clients

Cassandra Clients

CQLSH: Command-line tool for interacting with Cassandra using CQL.

DataStax Java Driver: High-performance driver supporting asynchronous queries and automated node discovery.

Python Driver (cassandra-driver): Standard client driver for Python applications.

Hadoop Integration

Cassandra integrates with Hadoop to perform analytical processing over operational NoSQL data:

InputFormat/OutputFormat: Custom Java classes (CqlInputFormat, CqlOutputFormat) allow Hadoop MapReduce jobs to read directly from and write back to Cassandra tables without transferring data over intermediate network hops.

Data Locality: MapReduce tasks are scheduled on the exact Cassandra node where the targeted partition data resides, reducing network I/O.

Spark/Hive/Pig Interoperability: Distributed engines use Cassandra connectors to process large datasets directly in-place.

Apache Pig

Apache Pig is a high-level data-flow platform that converts data pipelines written in Pig Latin into executable Hadoop MapReduce programs.

Grunt shells

The Grunt shell is the interactive command-line interface provided by Apache Pig. It allows users to write, test, and execute Pig Latin scripts sequentially and interactively. It serves as an essential tool for data engineers to inspect data and debug processing workflows prior to production batch execution.

Grunt is Pig's interactive shell.

- Interactive Mode: Execution via terminal commands using `pig -x local` (Local execution) or `pig -x mapreduce` (HDFS execution).
- Batch Mode: Execution of script files (`pig script.pig`).
- Key Utility Commands:
  - `ls` / `cd` – Browse HDFS file structures.
  - `sh` – Execute local Linux terminal commands.
  - `clear` – Clear shell view.

Pig Data Model

Pig supports nested, complex data structures:

Data Type	Description	Syntax Example
Atom	Single atomic value (Int, Float, String, Bytes).	'Nagpur', 101, 98.6
Tuple	Ordered collection of fields (equivalent to an RDBMS Row).	('Aarav', 24, 'Nagpur')
Bag	Unordered collection of tuples.	{('Aarav', 24), ('Priya', 22)}
Map	Set of Key-Value pairs (Key must be a character string).	['name'#'Aarav', 'role'#'Developer']

Pig Latin Scripting Mechanics

Pig scripts consist of data transformation statements called Relations.

LOAD: Reads data from storage.

FILTER: Filters rows based on boolean conditions.

FOREACH ... GENERATE: Transforms and projects columns.

GROUP: Groups records across matching key fields.

DUMP / STORE: Outputs results to terminal display or saves to HDFS storage.

Developing & Testing Pig Latin Scripts

Developers use diagnostic operators for testing and debugging during development:

DESCRIBE relation_name: Displays schema structures of relations.

EXPLAIN relation_name: Outlines logical, physical, and MapReduce execution plans.

ILLUSTRATE relation_name: Generates step-by-step sample data transformations to trace data flow logic without executing full MapReduce jobs.

Apache Hive

Apache Hive is an enterprise data warehouse infrastructure built on top of Apache Hadoop, providing data summarization, querying, and analysis using an SQL-like language called HiveQL.

Data Types and File Formats

Primitive Data Types: INT, BIGINT, FLOAT, DOUBLE, STRING, BOOLEAN, TIMESTAMP, DECIMAL.

Complex Data Types:

ARRAY<data_type>: Ordered collection (ARRAY['red', 'green'])

MAP<primitive_type, data_type>: Key-value mapping (MAP('key1', 10))

STRUCT<col_name : data_type, ...>: Record with named fields (STRUCT(name STRING, age INT)).

Storage File Formats

TextFile: Standard plain text format (delimited by commas or tabs); easy to view, but uses large storage space.

SequenceFile: Binary file format storing key-value pairs; supports compression.

ORC (Optimized Row Columnar): Columnar format that speeds up reads, compresses data efficiently, and supports ACID properties.

Parquet: Efficient columnar file structure widely used across Apache Spark and Big Data ecosystems.

HiveQL Data Definition Language (DDL)

Defines metadata schemas managed inside the Hive Metastore relational database.

```
-- Create an external managed Hive table

CREATE EXTERNAL TABLE IF NOT EXISTS student_logs (

    student_id INT,

    name STRING,

    courses ARRAY<STRING>,

    address MAP<STRING, STRING>

)

ROW FORMAT DELIMITED

FIELDS TERMINATED BY ','

COLLECTION ITEMS TERMINATED BY '$'

MAP KEYS TERMINATED BY ':'

STORED AS TEXTFILE

LOCATION '/data/mca/students';
```

HiveQL Data Manipulation (DML)

Handles loading, updating, and writing data pipeline transformations.

```
-- Loading raw HDFS file into Hive table
```

```
LOAD DATA INPATH '/hdfs/path/raw_data.csv' INTO TABLE
student_logs;

-- Inserting query output into a target table

INSERT OVERWRITE TABLE student_summary

SELECT student_id, name FROM student_logs WHERE student_id IS
NOT NULL;
```

HiveQL Queries

Supports standard SQL aggregations, joins, grouping, window functions, and partitioning/bucketing:

```
-- Grouping and Aggregation Query

SELECT

    address['city'] AS city,

    COUNT(student_id) AS total_students

FROM student_logs

GROUP BY address['city']

HAVING COUNT(student_id) > 5;
```